

WorksFree RPA 프로젝트

WorksFree 자동화 솔루션 모음

프로젝트 개요

WorksFree RPA 프로젝트는 기구설계자 및 제조업 엔지니어를 위한 Python 기반 업무 자동화 솔루션 모음입니다. 프로젝트는 세 가지 주요 카테고리로 구성되어 있습니다:

- 1. **10.common**: 모든 앱에서 공유하는 공통 모듈
- 2. **30.apps**: 외부 데스크톱 애플리케이션 자동화 스크립트
- 3. **50.data**: 순수 Python 데이터 처리 자동화 스크립트

최신 빌드 및 인증 현황 (2026-01-29)

빌드 버전

앱	최신 버전	빌드 시각	상태
BOM Exporter	1.0.2.6	2026-01-29 06:48	☒ 정상
Attribute Reset	0.7.2.9	2026-01-29 06:50	☒ 정상
DWG Batch Print	0.7.8.5	2026-01-29 06:48	☒ 정상
DWG Classifier	0.8.7.7	2026-01-29 06:47	☒ 정상
Conversion Verifier	0.8.3.8	2026-01-29 06:49	☒ 정상
Korean Filename Normalizer	0.8.4.4	2026-01-29 06:48	☒ 정상
QRCode Generator	0.7.2.2	2026-01-29 06:46	☒ 정상

인증 테스트 결과

테스트 일시: 2026-01-29 06:52-06:54

테스트 레벨: FULL (EXE 모드)

통합 리포트:

[90.tests/ui_lifecycle_test/test_results/certification_20260129_065258_exe/index.html](#)

앱	인증 등급	통과율	실행 시간	비고
BOM Exporter	 STANDARD	155/156 (99.4%)	17.1s	
DWG Batch Print	 STANDARD	155/156 (99.4%)	15.1s	
Attribute Reset	 STANDARD	155/156 (99.4%)	15.7s	
DWG Classifier	 STANDARD	155/156 (99.4%)	13.8s	
Conversion Verifier	 NONE	127/134 (94.8%)	12.8s	무료 앱 크레딧 로직
Korean Filename Normalizer	 NONE	127/134 (94.8%)	12.6s	무료 앱 크레딧 로직

앱	인증 등급	통과율	실행 시간	비고
QRCode Generator	⚠ NONE	127/134 (94.8%)	11.1s	무료 앱 크레딧 로직

유료 앱 (4개): STANDARD 등급 달성 ☒
무료 앱 (3개): 크레딧 테스트 로직 개선 필요 (핵심 기능은 정상)

번들링 표준화 현황

일관성: 100% 달성 ☒

- 7개 spec 파일 `prepare_user_configs()` 순서 통일
- glob 패턴 표준화 ('`silver-argon*.json`', '`worksfree-*.json`')
- 번들 구조 검증 자동화 (`verify_bundle_structure.ps1`)
- JSON 파일 숨김 속성 자동화 (`wf_app_init_helpers.set_json_files_hidden()`)

📁 앱 약어 (App Aliases)

프로젝트 내에서 사용하는 앱 약어:

약어	전체 이름	폴더 위치	크레딧 정책
be	BOM Exporter	30.apps/bom_exporter	100 크레딧/파일
dp	DWG Batch Print	30.apps/dwg_batch_print	10 크레딧/파일
dc	DWG Classifier	50.data/dwg_classifier	50 크레딧/작업
cv	Conversion Verifier	50.data/conversion_verifier	무료 앱 (trial_credits: -1)
kfn	Korean Filename Normalizer	50.data/korean_filename_normalizer	무료 앱 (trial_credits: -1)
ar	Attribute Reset	30.apps/attribute_reset	200 크레딧/작업
qr	QRCode Generator	50.data/qrcode_generator	무료 앱 (trial_credits: -1)

📁 사용자 환경 파일 구조

배포 환경 (사용자 PC)

C:\Users\{username}\.wf_rpa\	
├─ wf_rpa_config.json	# 전역: 사용자 등록 정보 (이메일, HW 지문, 최초 설치 앱)
├─ bom_exporter\	
│ └─ policy.json	# 앱별: 크레딧 정책 (작업당 차감량, 메모리/타임아웃 제한)
└─ credit_history.json	# 앱별: 크레딧 사용 이력 (잔여량, 로그, 충전 이력)
└─ settings.json	# 앱별: UI 설정 (마지막 경로, 옵션, 관리자 모드)

```

├─ dwg_batch_print\
│   └─ policy.json
│   └─ credit_history.json
│   └─ settings.json
├─ dwg_classifier\
│   └─ policy.json
│   └─ credit_history.json
│   └─ settings.json
├─ conversion_verifier\
│   └─ policy.json
│   └─ credit_history.json
│   └─ settings.json
├─ korean_filename_normalizer\
│   └─ policy.json
│   └─ credit_history.json
│   └─ settings.json
└─ attribute_reset\
    └─ policy.json
    └─ credit_history.json
    └─ settings.json

```

개발 환경

```

10.rpa/
├─ 10.common/
│   └─ config/
│       └─ wf_rpa_config.json
│       └─ silver-argon-*.json
│       └─ bom_exporter/
│           └─ policy.json
│           └─ credit_history.json
│           └─ settings.json
│       └─ dwg_batch_print/
│           └─ (동일 구조)
│       └─ dwg_classifier/
│           └─ (동일 구조)
│       └─ conversion_verifier/
│           └─ (동일 구조)
│       └─ korean_filename_normalizer/
│           └─ (동일 구조)
│       └─ attribute_reset/
│           └─ (동일 구조)
├─ 30.apps/{app}/
│   └─ logs/
└─ 50.data/{app}/
    └─ logs/

```

개발용 설정 통합 관리
전역: 사용자 등록 정보
Google Sheets 서비스 계정

앱별 크레딧 정책
앱별 크레딧 이력
앱별 UI 설정

앱별 로그 폴더 (30일 자동 삭제)

앱별 로그 폴더 (30일 자동 삭제)

참고: 개발 환경 설정 파일은 `10.common/config/` 폴더에 통합 관리됩니다. 개별 앱 폴더의 `config/` 폴더는 더 이상 사용하지 않습니다.

파일 역할 상세

파일명	위치	역할	주요 내용
<code>wf_rpa_config.json</code>	<code>.wf_rpa/</code> (루트)	사용자 전역 설정	이메일, HW 지문, 등록 상태, 최초 설치 앱, 구글 시트 연동 정보
<code>policy.json</code>	<code>.wf_rpa/{app}/</code>	앱별 크레딧 정책	<code>trial_credits</code> , <code>purchased_credits</code> , <code>credit_per_work</code> 등
<code>credit_history.json</code>	<code>.wf_rpa/{app}/</code>	앱별 사용 이력	잔여 크레딧, 사용 로그, 충전 이력, 마지막 동기화 시간
<code>settings.json</code>	<code>.wf_rpa/{app}/</code>	앱별 UI 설정	마지막 경로, UI 옵션, 관리자 모드, 로그 표시 여부

사용자 등록 흐름

1. **최초 설치 감지**: 6개 앱 중 어떤 앱이든 최초 실행 시 `wf_rpa_config.json`의 `is_registered` 값 확인
2. **등록 버튼 표시**: 미등록 상태(`is_registered: false`)면 "등록" 버튼 활성화
3. **등록 정보 수집**: 이메일 + 하드웨어 지문(CPU/메인보드 시리얼) 수집
4. **전역 설정 저장**: `wf_rpa_config.json`에 등록 정보 저장 (모든 앱에서 공유)
5. **이후 앱 설치**: 두 번째 이후 설치되는 앱은 기존 등록 정보 자동 로드, 등록 버튼 숨김
6. **Google Sheets 동기화**: 등록 완료 시 구글 시트에 사용자 정보 업로드, 정책 다운로드

Legacy 파일 마이그레이션

자동으로 이전 파일명을 감지하여 새 파일명으로 변환합니다:

- `dev_wf_rpa_config.json` → `wf_rpa_config.json`
- `.wf_rpa_config.json` → `wf_rpa_config.json` (숨김 파일 제거)
- `{app}.app_policy.json` → `credit_policy.json`
- `.{app}_credits.json` → `credit_history.json`
- `dev_{app}_settings.json` → `settings.json`

📁 프로젝트 구조

10.common/ - 공통 모듈

모든 앱에서 공유하는 핵심 기능 모듈입니다. 앱 간 일관성과 코드 재사용성을 보장합니다.

주요 모듈:

- `wf_log.py` - 통합 로깅 시스템 (싱글톤 패턴, 30일 자동 정리)
- `wf_credit_manager.py` - 크레딧 관리 및 사용량 추적
- `wf_register.py` - 사용자 등록 및 인증 (하드웨어 지문 기반)
- `wf_email.py` - 이메일 발송 기능
- `wf_googlesheets_manager.py` - Google Sheets 연동
- `wf_hwinf.py` - 하드웨어 정보 수집 (CPU/메인보드 지문)
- `wf_license.py` - 라이선스 검증

- `wf_app_init_helpers.py` - 앱 초기화 공통 헬퍼 함수
- `wf_settings_common.py` - 설정 파일 로드/저장 공통 유틸리티

공통 설정 파일 (config/):

개발 환경에서 모든 앱이 공유하는 설정 파일들:

- `config/wf_rpa_config.json` - 전역 사용자 정보 (등록 상태, 이메일, HW 지문)
- `config/{app}/policy.json` - 앱별 크레딧 정책
- `config/{app}/settings.json` - 앱별 UI 설정
- `config/{app}/credit_history.json` - 앱별 크레딧 사용 이력
- `config/silver-argon-*.json` - Google Sheets 서비스 계정 자격증명

30.apps/ - 외부 앱 자동화 (3rd-party Application Automation)

SolidWorks, AutoCAD, MS Office 등 외부 데스크톱 애플리케이션을 직접 구동하여 그 UI/COM/API를 통해 작업을 수행하는 자동화 스크립트를 관리하는 영역입니다.

특징:

- 실행 방식: 대상 앱 프로세스 제어 (키보드/마우스/COM/Win32 API)
- 요구사항: 대상 앱 설치/라이선스 필수
- UI 포커스/해상도 등에 민감
- 목적: "앱을 통해 데이터를 가공"하는 자동화

권장 규칙:

1. 자동화 로직은 `automation.py`, GUI는 `ui_main.py`로 분리
2. 외부 앱 연결/핸들 획득은 재시도(backoff) 로직 포함
3. 좌표 하드코딩 지양, 컨트롤 탐색 기반(pywinauto) 우선
4. 실패/대기 상황을 사용자 친화적 로그로 안내

현재 앱:

1. **bom_exporter** (약어: **be**) ☒

- 기능: SolidWorks BOM을 Excel로 자동 변환
- 상태: 배포 준비 완료
- 크레딧: 100 크레딧/파일
- 주요 파일: `ui_main.py`, `automation.py`, `bom_exporter.spec`
- 패키징: NSIS 인스톨러 + 포터블 버전

2. **dwg_batch_print** (약어: **dp**) ☒

- 기능: DWG 파일 일괄 인쇄 자동화
- 상태: 배포 준비 완료
- 크레딧: 10 크레딧/파일
- 주요 파일: `ui_main.py`, `automation.py`, `dwg_batch_print.spec`
- 패키징: NSIS 인스톨러 + 포터블 버전

3. **attribute_reset** (약어: **ar**) ☒

- 기능: DWG 파일 속성(Attribute) 일괄 초기화
- 상태: 배포 준비 완료
- 크레딧: 200 크레딧/작업
- 주요 파일: `ui_main.py`, `automation.py`, `attribute_reset.spec`
- 패키징: NSIS 인스톨러 + 포터블 버전

50.data/ - 순수 Python 데이터 처리 (Pure Python Data Apps)

외부 3rd-party GUI 애플리케이션을 직접 구동하지 않고, 파이썬 코드만으로 파일/텍스트/구조화 데이터 등을 처리·정규화·검증하는 앱입니다.

특징:

- 실행 방식: 표준 라이브러리/경량 파이썬 패키지 중심 처리
- 장점: 재현성·이식성 높고 CI/CD·자동 테스트에 적합
- 목적: "파이썬 내에서 데이터를 직접 가공"하는 자동화

권장 규칙:

1. 외부 GUI/프로세스 제어 코드는 포함하지 않음 (필요 시 30.apps로 이관)
2. I/O 경계와 순수 변환 로직 분리, 단위테스트 용이성 확보
3. 크레딧/정책 필요 시 `wf_credit_manager` 통합

현재 앱:

1. **dwg_classifier** (약어: **dc**) ☒

- 기능: 엑셀 규칙 기반 DWG 파일 자동 분류 및 정리
- 상태: 배포 준비 완료 (integrated spec, NSIS installer, portable)
- 크레딧: 50 크레딧/작업
- 주요 파일: `ui_main.py`, `automation.py`, `dwg_classifier.spec`, `README.md`
- 패키징: NSIS 인스톨러 + 포터블 버전

2. **korean_filename_normalizer** (약어: **kfn**) ☒

- 기능: 자소 분리된 한글 파일명 자동 감지 및 정규화
- 상태: 배포 준비 완료 (integrated spec, NSIS installer, portable)
- 크레딧: 무료 앱 (체험판 크레딧 -1)
- 주요 파일: `ui_main.py`, `automation.py`, `korean_filename_normalizer.spec`, `README.md`
- 패키징: NSIS 인스톨러 + 포터블 버전

3. **conversion_verifier** (약어: **cv**) ☒

- 기능: 변환 전후 파일 무결성 검증 및 차이 분석
- 상태: 배포 준비 완료 (integrated spec, NSIS installer, portable)
- 크레딧: 무료 앱 (trial_credits: -1)
- 주요 파일: `ui_main.py`, `automation.py`, `conversion_verifier.spec`, `README.md`
- 패키징: NSIS 인스톨러 + 포터블 버전

4. **qrcode_generator** (약어: **qr**) ☒

- 기능: QR 코드 일괄 생성 및 관리

- 상태: 배포 준비 완료 (integrated spec, NSIS installer, portable)
- 크레딧: 무료 앱 (trial_credits: -1)
- 주요 파일: `ui_main.py`, `automation.py`, `qrcode_generator.spec`, `README.md`
- 패키징: NSIS 인스톨러 + 포터블 버전

70.webs/ - 웹 관련

웹 인터페이스 및 데모

90.tests/ - 테스트 및 인증

단위 테스트, 통합 테스트, 앱 인증 도구를 포함합니다.

테스트 구조:

- `10.common/`: 공통 모듈 테스트
- `30.apps/`: 앱별 테스트
- `ui_lifecycle_test/`: **WF-ACT 인증 툴킷**

WF-ACT (WF-RPA App Certification Toolkit)

앱 라이프사이클 전체를 자동으로 인증하는 통합 테스트 도구입니다.

인증 레벨:

레벨	테스트 수	설명
BASIC	~40개	핵심 기능 (필수 통과)
STANDARD	~60개	일반 시나리오
FULL	166개+	모든 엣지케이스 (32개 정적 + 134개 동적)

사용법:

```
cd 90.tests/ui_lifecycle_test

# 모든 앱, FULL 레벨 인증 (권장)
python run_certification.py --level full

# 특정 앱만 인증
python run_certification.py --app bom_exporter --level full
python run_certification.py --app be dc ar --level standard

# 정적 인증만 실행 (빠른 코드 검증)
python run_static_certification.py --app bom_exporter

# 테스트 목록 확인
python run_certification.py --list

# EXE 패키지 인증 (배포 전 검증)
python run_certification.py --exe --level full
```

앱 약어:

- **be**: bom_exporter
- **dp/dbp**: dwg_batch_print
- **dc**: dwg_classifier
- **cv**: conversion_verifier
- **kfn**: korean_filename_normalizer
- **ar**: attribute_reset
- **qr**: qrcode_generator

테스트 스위트 (8개):

1. **ConfigSuite** - 설정 관리 (policy.json, settings.json, Google Sheets 연결성)
2. **CreditsSuite** - 크레딧 시스템 (차감, 동기화, 무료앱 처리)
3. **LifecycleSuite** - 앱 라이프사이클 (시작, 종료, 상태 관리)
4. **RegistrationSuite** - 사용자 등록 (이메일, HW 지문)
5. **SettingsSuite** - UI 설정 (geometry, topmost, 저장/로드)
6. **UISuite** - UI 기능 (버튼, 단축키, 관리자 모드)
7. **VersionSuite** - 버전 관리 (형식, 최소 요구사항)
8. **StaticAnalysisSuite** - 정적 코드 분석 (구조, 패턴 검증)

출력 결과:

```
test_results/certification_YYYYMMDD_HHMMSS/
├─ index.html          # 통합 웹 리포트 (모든 앱 요약)
├─ certification_result.json
├─ {app_name}_report.html # 앱별 상세 리포트
└─ {app_name}_result.json # 앱별 JSON 결과
```

인증 등급:

-  FULL: 모든 테스트 통과
-  STANDARD: STANDARD 레벨까지 통과
-  BASIC: BASIC 레벨만 통과
-  NONE: BASIC 미달

공통 아키텍처 패턴

크레딧 시스템

모든 앱은 통합 크레딧 시스템을 사용합니다:

- **체험판 (trial_credits)**: 기본 2,000 크레딧 제공 (무료 앱은 -1)
- **구매 크레딧 (purchased_credits)**: Google Sheets 연동으로 구매 내역 관리
- **무료 앱**: **trial_credits**: -1 (kfn, cv 등)
- **무제한 라이선스**: **purchased_credits**: -1 (영구 라이선스 구매 사용자)

크레딧 타입별 동작:

trial_credits	purchased_credits	상태	UI 표시
-1	(무시)	무료 앱	"무료"
양수	-1	무제한	"무제한"
양수	양수	유료	잔여 크레딧 표시
0	0	크레딧 없음	"크레딧 없음"

사용 로그 동기화:

- 무료 앱이든 무제한이든 **모든 사용 로그는 Google Sheets에 동기화**
- 사용량 추적 및 통계 목적으로 사용

개발/배포 모드 자동 감지**개발 모드:**

- `sys.argv[0]`가 `.py` 파일이거나 `WF_RPA_DEV=1` 환경변수 설정 시
- `10.common/config/` 디렉토리에서 설정 파일 로드
- 예: `10.common/config/bom_exporter/settings.json`

배포 모드:

- PyInstaller로 빌드된 실행 파일 실행 시
- 사용자 홈폴더(`~/ .wf_rpa`) 사용
- 모든 설정 파일이 사용자 홈에 생성/참조됨

로깅 시스템**wf_log.py 통합 로깅:**

- 싱글톤 패턴으로 앱 전체에서 단일 로거 인스턴스 사용
- 파일 + 콘솔 이중 출력
- 로그 파일 위치: `{app_folder}/logs/YYYYMMDD.txt`
- **30일 경과 로그 자동 삭제** (`_clean_old_logs()` 함수)

로그 파일 예시:

```
10.rpa/30.apps/bom_exporter/logs/20260116.txt
10.rpa/50.data/dwg_classifier/logs/20260116.txt
```

관리자 모드 (Admin Mode)**기능:**

- 관리자 비밀번호 입력 시 상세 로그 패널 표시
- **하드웨어 정보 로깅:** 관리자 모드 전환 시 CPU ID, 메인보드 ID, 하드웨어 지문을 로그에 출력

- 6개 앱 모두 wf_hwinfo 모듈을 사용하여 HW 정보 수집 구현 완료

HW 정보 로깅 출력 예시:

```
-----
하드웨어 정보
-----
하드웨어 지문: ABC123DEF456...
CPU ID: BFEBFBFF000906A3
메인보드 ID: 123456789
```

로딩 최적화

목표: UI 로딩 시간 1초 이하 달성

적용 기법:

- 1. **Lazy Import**: 무거운 모듈(automation, pywinauto 등)은 버튼 클릭 시 로드
- 2. **Background Policy Loading**: 정책 파일 비동기 로드
- 3. **File Caching**: mtime 기반 캐시로 불필요한 JSON 파싱 제거
- 4. **Startup Profiling**: `_STARTUP_ENABLED` 플래그로 로딩 시간 측정

앱별 최적화 현황:

앱	Startup Profiling	Lazy Import	비고
bom_exporter	☑	☑	가장 무거운 앱, 1초 이하 달성
dwg_batch_print	☑	☑	pywinauto lazy load
dwg_classifier	☑	☑	-
conversion_verifier	☑	☑	-
korean_filename_normalizer	☑	☑	-
attribute_reset	-	☑	경량 앱, profiling 불필요

Startup Profile 로그 위치:

- 개발 모드: `{app_folder}/startup_profile.log`
- 배포 모드: `~/.wf_rpa/{app}/startup_profile.log`

 빌드 및 배포

빌드 요구사항

- Python 3.13.7
- PyInstaller 6.16.0
- NSIS 3.11 (인스톨러 생성용)
- Windows 11

빌드 구조

각 앱 폴더별 빌드 스크립트

각 앱 폴더에는 전용 빌드 스크립트가 포함되어 있습니다:

```

  30.apps/bom_exporter/
    [x] build_bom_exporter.ps1      # 개별 빌드 스크립트
    [x] bom_exporter.spec          # PyInstaller 설정

  30.apps/dwg_batch_print/
    [x] build_dwg_batch_print.ps1
    [x] dwg_batch_print.spec

  50.data/dwg_classifier/
    [x] build_dwg_classifier.ps1
    [x] dwg_classifier.spec

  50.data/conversion_verifier/
    [x] build_conversion_verifier.ps1
    [x] conversion_verifier.spec

  50.data/korean_filename_normalizer/
    [x] build_korean_filename_normalizer.ps1
    [x] korean_filename_normalizer.spec

  50.data/attribute_reset/
    [x] build_attribute_reset.ps1
    [x] attribute_reset.spec

```

루트 폴더 (10.rpa)

```

  10.rpa/
    [x] build_all.ps1              # 전체 6개 앱 일괄 빌드 (be, dp, dc, cv, kfn,
ar)
    [x] build_all_parallel.ps1    # 병렬 빌드
    [x] test_loading_time.ps1     # 앱 로딩 시간 테스트 (3초 목표)
    [x] check_build_environment.ps1 # 빌드 전 환경 검증

```

빌드 방법

권장 빌드 절차 (3단계)

1단계: 환경 검증 (필수)

```

cd D:\drive_files\10.worksfree\10.rpa
.\check_build_environment.ps1

```

- 실행 중인 앱 프로세스 자동 감지 및 종료 옵션
- Python 3.13.7 및 PyInstaller 6.16.0 확인
- 디스크 여유 공간 확인 (5GB 이상 권장)
- 기존 빌드 폴더 상태 점검

2단계: 개별 앱 빌드 (권장)

```
# 각 앱 폴더에서
cd 30.apps/bom2excel
.\build_bom2excel.ps1

cd 50.data/conversion_verifier
.\build_conversion_verifier.ps1

cd 50.data/dwg_classifier
.\build_dwg_classifier.ps1

cd 50.data/korean_filename_normalizer
.\build_korean_filename_normalizer.ps1
```

3단계: 전체 앱 일괄 빌드 (선택)

```
# 10.rpa 폴더에서
cd 10.rpa
.\build_all.ps1
```

빌드 시스템 개선사항 (2024-11-11) ☒

dist 폴더 문제 재발방지 대책

모든 빌드 스크립트에 **3단계 폴더 정리 메커니즘** 추가:

1. **1차 시도**: 일반 삭제 (**Remove-Item -Recurse -Force**)
2. **2차 시도**: 관련 프로세스 종료 후 재삭제
3. **3차 시도**: 하위 파일 개별 삭제 후 폴더 삭제
4. **Fallback**: 실패해도 경고만 출력하고 빌드 계속 진행

핵심 기능:

- Windows 파일 락 자동 해제
- 백그라운드 프로세스 자동 종료
- 빌드 전 환경 검증 스크립트
- 실패 시에도 빌드 계속 진행 (PyInstaller가 덮어쓰기)

테스트 결과: 3개 앱 연속 빌드 모두 성공 (오류 없음)

빌드 결과물

각 빌드는 자동으로 다음을 생성합니다:

```
D:\release\candidates\{app}_{timestamp}/
├── {app}_{version}_installer.exe      # NSIS 인스톨러
├── {app}_portable/                   # 포터블 버전 폴더
│   ├── {app}.exe
│   ├── _internal/                   # 의존성 파일들
│   └── ...
├── {app}_portable.zip                # 포터블 버전 압축
├── metadata/
│   └── build_info.json               # 빌드 메타데이터
```

버전 관리 스키마

- **단일 소스:** 각 앱의 `config/{app}/settings.json`의 `app_config.full_version`에서 버전을 읽음
- **버전 포맷:** `v{MAJOR}.{MINOR}.{PATCH}.{BUILD}` (예: `v0.7.0.0`, `v0.7.0.3`)
- **증가 규칙:** 빌드 시 PyInstaller spec의 `load_and_increment_version()` 함수가:
 1. BUILD(우측끝)을 +1 증가
 2. BUILD이 10을 초과하면 0으로 리셋하고 PATCH(그 좌측)을 +1
 3. PATCH이 10을 초과하면 0으로 리셋하고 MINOR를 +1
 4. 변경된 버전을 즉시 `settings.json`에 저장
- **초기 버전:** `settings.json`이 없거나 빈 값이면 기본값 `v0.7.0.0`에서 시작 → 첫 빌드 산출물은 `v0.7.0.1`
- **표시 규칙:**
 - `APP_VERSION_FULL`: 4자리 버전 (0.7.0.x) - 관리자 모드 및 상세 정보에 사용
 - `APP_VERSION_DISPLAY`: 앞 2자리 (v0.7) - 일반 사용자용 UI/타이틀에 사용

성능 최적화 (2024-11-11 완료) ☒

BOM2Excel 최적화 상세

적용 기법:

1. **Logger Lazy Loading:** `@property` 패턴으로 첫 사용 시점에 로딩
2. **Background Policy Loading:** `_load_config_fast()` + `load_policies_async()`
3. **File Caching:** mtime 기반 캐시로 불필요한 JSON 파싱 제거

성능 개선:

- Config 로딩: 1363ms → 6ms (99.6% 개선)
- 전체 로딩: 2446ms → 1046ms (57% 개선, 1.4초 단축)

DWG Classifier 최적화

적용 기법:

1. **Logger Lazy Loading:** BOM2Excel과 동일한 패턴 적용

2. **File Naming Cleanup:** . 접두어 제거 (`.dwg_classifier_settings.json` → `dwg_classifier_settings.json`)

예상 성능 개선: ~1초

Conversion Verifier & Korean Filename Normalizer

- **최적화 불필요:** wf_log를 사용하지 않아 logger 초기화 오버헤드 없음
- **현재 성능:** 양호

로딩 시간 테스트

```
# 10.rpa 폴더에서
.\test_loading_time.ps1
```

자동으로 최신 빌드를 찾아 4개 앱을 각각 3회씩 테스트하고 통계를 출력합니다.

문제 발생 시 대응

즉시 조치

```
# 1. 모든 앱 프로세스 강제 종료
Get-Process | Where-Object {
    $_.Name -like "*bom2excel*" -or
    $_.Name -like "*dwg_classifier*" -or
    $_.Name -like "*conversion_verifier*" -or
    $_.Name -like "*korean_filename_normalizer*"
} | Stop-Process -Force

# 2. 빌드 스크립트 재실행
.\build_xxx.ps1
```

여전히 실패 시

상세한 트러블슈팅은 [BUILD_TROUBLESHOOTING.md](#) 참고

📦 배포 패키징 표준 (Packaging Standards)

1. 성능 목표 (Performance Targets)

항목	목표	설명
시작 속도	< 3초 (릴리스)	onedir 구조 + 불필요한 대형 라이브러리 제외(PyQt5, matplotlib 등)
아이콘 로딩	< 50ms	<code>ensure_{app}_icon()</code> 에서 16x16 32-bit BGRA 직접 생성
메모리 초기 점유	최소화	lazy import (예: 무거운 automation 모듈은 UI 이벤트 후 로딩)

항목	목표	설명
로그 초기화	< 100ms	<code>wf_log</code> 싱글톤 + <code>propagate=False</code> 중복 제거

2. 환경 필수요건 (Environment Prerequisites)

- Python 3.13.7 (PATH에 하위 버전 `pyinstaller.exe` 잔존 금지)
- PyInstaller 6.16.0 (실행은 `python -m PyInstaller` 방식 고정)
- NSIS 3.11 (경로: `C:\Program Files (x86)\NSIS\makensis.exe`)
- UTF-8 콘솔 환경 (spec 상단 `TextIOWrapper` 재설정)

3. 출력 구조 (Output Conventions)

```
D:\release\candidates\{app}_{YYYYMMDD_HHMMSS}/
  {app}_{version}_installer.exe
  {app}_{version}_portable/ (포터블 폴더)
  {app}_{version}_portable.zip
```

- 아이콘: `res/{APP}.ico`
- NSIS 스크립트: `{app}_installer.nsi` (UTF-8 BOM, `encoding='utf-8-sig'`)
- Portable 실행 배치(Optional): `run_{app}.bat` (필요 시 cp949 인코딩)

4. Spec 템플릿 필수 블록

1. `ensure_{app}_icon()` – 외부 라이브러리 없이 바이너리 아이콘 생성
2. 앱 메타정보: `APP_NAME`, `APP_VERSION`, `APP_DISPLAY_NAME`, `APP_DESCRIPTION`, `APP_PUBLISHER`
3. 플래그: `DEBUG_BUILD`, `STARTUP_PROFILING`
4. 자원/경로: `SPEC_DIR`, `WORKSPACE_ROOT`, `BUILD_OUTPUT_DIR`
5. 정책 생성: `create_global_policies()` (중복 키 금지 – 최근 dwg/kfn 수정 사례 참고)
6. 사용자 홈 준비: `prepare_user_configs()` – 전역 설정/credentials 배치
7. 히든 임포트: `get_optimized_hidden_imports()` – 앱별 최소 모듈만 추가
8. NSIS 스크립트 생성: `create_nsis_script()` – LICENSE 페이지 선택적(; 주석), 아이콘 경로 `res/*.ico`
9. 빌드 후 자동화: `post_build_automation()` – NSIS 실행 + portable 패키징 + zip + 정리
10. EXE 생성 전 아이콘 미리 생성: `icon_path = ensure_{app}_icon()`
11. Windows에서 `strip=False` (`strip` 도구 부재로 경고 회피)

5. 히든 임포트 카테고리 (Hidden Imports Categories)

- WorksFree Core: `wf_log`, `wf_credit_manager`, `wf_app_base`, `wf_register`
- GUI Core: `tkinter`, `tkinter.ttk`, `tkinter.messagebox`, `tkinter.filedialog`
- Automation (필요 시): `pyautogui`, `pyscreeze`, `pymsgbox`, `pytweening`
- 시스템/유틸: `unicodedata`, `re`, `pathlib`, `shutil`, `argparse`, `time`, `threading`, `datetime`, `json`
- 런타임 필수: `zipfile`, `multiprocessing`
- 앱 특화 예시:
 - `dwg_classifier`: `glob`, `tempfile`
 - `korean_filename_normalizer`: `unicodedata`, `re`, `glob`

- `conversion_verifier: difflib, filecmp`
- 플랫폼 확장(필요 시): `pywin32` (`win32api`, `win32con`, `win32timezone` 등), `pywinauto` (`application`, `findwindows`, `controls.*`), `comtypes`

6. 수평전개(Horizontal Deployment) 절차

1. 대상 앱 폴더 진입 (`50.data/{app}` 또는 `30.apps/{app}`)
2. 기존 기본 spec 백업 (`{app}.spec.old`)
3. 템플릿(spec) 복사 후 앱 메타정보 치환
4. 아이콘 함수 색상/이름 변경 (예: B2E -> DWG -> KFN)
5. 중복 정책 키 제거 및 앱별 credit/memory 설정 조정
6. 히든 임포트 최소화 (중복 및 불필요 라이브러리 제거)
7. NSIS 스크립트: LICENSE 페이지 필요 없으면 주석 처리
8. 프로세스 종료 로직 단순화 (`taskkill /F /IM {app}.exe /T`)
9. 변수 참조 순서 점검 (예: `portable_base_dir` 정의 후 사용) → `dwg_classifier`에서 발생했던 오류 예방
10. 빌드 명령 실행: `python -m PyInstaller {app}.spec --noconfirm --log-level=WARN`
11. 결과 폴더/인스톨러/portable/zip 검증 → 실행 테스트
12. README 및 기록 로그 업데이트

7. NSIS Uninstall 규칙 (표준)

- 실행 중 프로세스 강제 종료 (`taskkill /F /IM {app}.exe /T`)
- 설치 폴더 전부 제거 (`RMDir /r "$INSTDIR"`)
- 레지스트리 제거 (HKLM `Software\WorksFree\{app}` 및 Uninstall 키)
- 바로가기 제거 (사용자/공용 시작 메뉴)
- 사용자 데이터 선택적 정리 (환경 보존 옵션 허용 가능)

8. 오류 회피 가이드 (Common Pitfalls)

- NSIS: `StrContains` 커스텀 매크로 불필요 → 단순 `taskkill` 로직 사용
- 인코딩: NSIS 스크립트는 `utf-8-sig`로 저장 (Bad text encoding 방지)
- 아이콘: NSIS와 PyInstaller 모두 `res\{APP}.ico` 존재 필수 (생성 함수 사전 호출)
- 변수 순서: 후처리 자동화에서 참조 전에 정의 (`portable_base_dir` 사례)
- strip: Windows에서는 `strip=False` (경고/오류 회피)
- `pyinstaller.exe` (구버전) 호출 금지 → 항상 `python -m PyInstaller`

9. 성능 최적화 패턴

- Lazy Import: GUI 이벤트 시 무거운 모듈 로딩 (예: 버튼 클릭 시 `import automation`)
- 모듈 제외: 사용하지 않는 대형 패키지(PyQt5, matplotlib, scipy, torch 등) `excludes` 처리
- 로그 경량화: `DEBUG_BUILD=False` 시 콘솔 비활성화, 파일 로그 유지
- 리소스 최소화: 필수 파일만 `datas`로 수집 (정책 / credentials / 아이콘)

10. 새 앱 Onboarding Quick Start

- 1) 기존 최적화된 spec 복사 → 이름 치환
- 2) `ensure_{app}_icon()` 색상/코드 조정
- 3) 히든 임포트 앱별 최소화

- 4) NSIS 생성/portable 자동화 블록 유지

5) 빌드 실행 및 결과 검증

6) README 로드맵/수평전개 목록 갱신

공통 개선사항

완료된 개선

1. 통합 로깅 시스템
 - 싱글톤 패턴, 모듈명 표시
 - 파일/콘솔 이중 출력
 - 30일 자동 정리
2. 패키징 최적화
 - PIL/Pillow 포함 (PyAutoGUI 의존성)
 - pywin32 모듈 완전 포함 (win32timezone 등)
 - pywinauto 서브모듈 완전 포함
 - comtypes 포함
3. NSIS 인스톨러 개선
 - 프로세스 자동 종료
 - 상세한 제거 과정 안내
 - 사용자 데이터 선택적 제거
 - 다중 앱 지원
4. 빌드 플래그
 - `DEBUG_BUILD` - 콘솔 활성화 제어
 - `STARTUP_PROFILING` - 시작 시간 측정

앱 현황

앱 이름	약어	상태	Spec	크레딧/작업	설명
bom_exporter	be	☒ 완료	integrated	100	SolidWorks BOM → Excel
dwg_batch_print	dp	☒ 완료	integrated	10	DWG 일괄 인쇄
dwg_classifier	dc	☒ 완료	integrated	50	DWG 파일 분류
conversion_verifier	cv	☒ 완료	integrated	무료 (-1)	변환 무결성 검증
korean_filename_normalizer	kfn	☒ 완료	integrated	무료 (-1)	한글 파일명 정규화
attribute_reset	ar	☒ 완료	integrated	200	DWG 속성 초기화

개발 가이드

새 앱 추가 시 체크리스트

1. ☐ wf_log 통합 로깅 사용
2. ☐ wf_app_base 기반 구조 적용 (해당되는 경우)
3. ☐ PyInstaller spec 파일 작성
4. ☐ NSIS 인스톨러 스크립트 포함
5. ☐ 테스트 케이스 작성 (90.tests/)
6. ☐ README.md 업데이트

코딩 규칙

- Python 3.13+ 표준 준수
- UTF-8 인코딩 필수
- 로깅은 wf_log.get_app_logger() 사용
- 크레딧 시스템 통합 필수 (해당되는 경우)

기술 스택

- 언어: Python 3.13.7
- GUI: Tkinter
- 자동화: pywinauto (UIA 백엔드), pyautogui
- 데이터 처리: pandas, openpyxl
- 빌드: PyInstaller 6.16.0
- 인스톨러: NSIS 3.11
- 플랫폼: Windows OS

배포 형상 테스트 시나리오

배포 패키지가 정상 동작하는지 검증하기 위한 테스트 시나리오입니다.

테스트 환경

```
테스트 위치: D:\release\candidates\{앱명}_{버전}\{앱명}_{버전}_portable\  
빌드 명령: .\build_{앱명}.ps1 -BuildType 2
```

1. 기본 실행 테스트

단계	확인 항목	예상 결과
1	EXE 더블클릭	스플래시 없이 메인 UI 표시
2	메인 윈도우 표시	3초 이내 UI 완전 로딩
3	앱 종료 (X 버튼)	프로세스 완전 종료 (작업관리자 확인)

2. 신규 사용자 등록 플로우 (최초 설치)

사전 조건: C:\Users\{username}\.wf_rpa\ 폴더가 없는 상태 (또는 wf_rpa_config.json 삭제)

단계	확인 항목	예상 결과
1	앱 실행	미등록 상태로 인식
2	UI 확인	"등록" 버튼 표시됨
3	등록 버튼 클릭	이메일 입력 다이얼로그 표시
4	이메일 입력 후 확인	Google Sheets 동기화 시작
5	등록 완료	성공 메시지, "등록" 버튼 숨김
6	wf_rpa_config.json 확인	is_registered: true, user_email 저장됨

3. 기존 사용자 (두 번째 앱 설치)

사전 조건: 첫 번째 앱에서 등록 완료된 상태

단계	확인 항목	예상 결과
1	다른 앱 실행	기존 등록 정보 자동 로드
2	UI 확인	"등록" 버튼 표시되지 않음
3	크레딧 상태	Google Sheets에서 정책 동기화 완료

4. 크레딧 시스템 검증

4.1 유료 앱 (be, dp, dc, ar)

단계	확인 항목	예상 결과
1	초기 상태	trial_credits 표시 (예: 2000)
2	작업 수행	크레딧 차감 (credit_per_work × 처리 건수)
3	잔여 크레딧 확인	UI에 정확한 잔여량 표시
4	Google Sheets 확인	사용 로그 동기화됨
5	크레딧 0일 때	작업 차단 + 안내 메시지

4.2 무료 앱 (kfn, cv) - trial_credits: -1

단계	확인 항목	예상 결과
1	UI 상태	"무료" 표시
2	작업 수행	크레딧 차감 없이 정상 동작
3	Google Sheets 확인	사용 로그는 계속 동기화됨

4.3 무제한 사용자 - purchased_credits: -1

단계	확인 항목	예상 결과
1	UI 상태	"무제한" 표시
2	작업 수행	크레딧 차감 없이 정상 동작
3	Google Sheets 확인	사용 로그는 계속 동기화됨

5. 설정 파일 경로 검증

파일	경로	확인 항목
전역 설정	~\.wf_rpa\wf_rpa_config.json	사용자 정보, 이메일 설정
앱 정책	~\.wf_rpa\{app}\policy.json	크레딧 정책 (번들에서 복사됨)
사용 이력	~\.wf_rpa\{app}\credit_history.json	잔여 크레딧, 사용 로그
UI 설정	~\.wf_rpa\{app}\settings.json	마지막 경로, UI 옵션

6. 오프라인 동작 검증

단계	확인 항목	예상 결과
1	네트워크 차단 후 앱 실행	로컬 캐시로 정상 실행
2	작업 수행	로컬 크레딧으로 동작
3	동기화 실패	경고 메시지 후 계속 진행
4	네트워크 복구 후	다음 실행 시 동기화 재시도

7. 앱별 핵심 기능 테스트

앱	테스트 시나리오	확인 항목
bom_exporter	SolidWorks 어셈블리 → Excel	BOM 추출, Excel 저장, 크레딧 차감
dwg_batch_print	DWG 폴더 선택 → 인쇄	파일 목록, 인쇄 실행, 크레딧 차감
dwg_classifier	분류 규칙 + DWG 폴더	파일 분류, 이동/복사, 크레딧 차감
conversion_verifier	원본/변환 폴더 비교	무결성 검증, 결과 리포트
korean_filename_normalizer	자소 분리 파일 폴더	미리보기, 정규화 실행
attribute_reset	DWG 파일 선택	속성 초기화 실행

8. 패키지 무결성 체크리스트

- {앱명}.exe 실행 가능
- _internal\ 폴더 존재
- _internal\.wf_rpa\{앱명}\policy.json 존재
- _internal\.wf_rpa\wf_rpa_config.json 존재
- create_desktop_shortcut.bat 동작 (바로가기 생성)

9. 빠른 테스트 스크립트

```
# 모든 앱 빌드 후 candidates 폴더에서 테스트
.\build_all.ps1 -BuildType 2

# 개별 앱 빌드
cd 30.apps\bom_exporter
.\build_bom_exporter.ps1 -BuildType 2

cd 30.apps\dwg_batch_print
.\build_dwg_batch_print.ps1 -BuildType 2

cd 50.data\dwg_classifier
.\build_dwg_classifier.ps1 -BuildType 2

cd 50.data\conversion_verifier
.\build_conversion_verifier.ps1 -BuildType 2

cd 50.data\korean_filename_normalizer
.\build_korean_filename_normalizer.ps1 -BuildType 2

cd 50.data\attribute_reset
.\build_attribute_reset.ps1 -BuildType 2
```

10. Geometry 저장 기능 테스트

모든 창(메인창, 설정창, 등록창)에서 Alt+G로 geometry를 settings.json에 저장합니다.

테스트 절차

단계	창 종류	테스트 절차	예상 결과
1	메인창	앱 실행 → 창 이동/크기 조정 → Alt+G	토스트 "geometry 저장됨: WxH+X+Y" 표시
2	설정창	설정 버튼 클릭 → 창 이동 → Alt+G	토스트 "geometry 저장됨: WxH+X+Y" 표시
3	등록창	체험판 등록 열기 → 창 이동 → Alt+G	토스트 "geometry 저장됨: WxH+X+Y" 표시
4	검증	settings.json 열어서 확인	ui_config 섹션에 3개 geometry 값 존재

settings.json 저장 구조

```
{
  "ui_config": {
    "window_geometry_override": "800x600+100+100",
    "settings_window_geometry": "600x400+150+150",
    "registration_window_geometry": "500x350+200+200"
```

```
}
}
```

앱별 테스트 체크리스트

- bom_exporter
 - 메인창 Alt+G → window_geometry_override 저장
 - 설정창 Alt+G → settings_window_geometry 저장
 - 등록창 Alt+G → registration_window_geometry 저장
- dwg_batch_print
 - 메인창 Alt+G → window_geometry_override 저장
 - 설정창 Alt+G → settings_window_geometry 저장
 - 등록창 Alt+G → registration_window_geometry 저장
- dwg_classifier
 - 메인창 Alt+G → window_geometry_override 저장
 - 설정창 Alt+G → settings_window_geometry 저장
 - 등록창 Alt+G → registration_window_geometry 저장
- conversion_verifier
 - 메인창 Alt+G → window_geometry_override 저장
 - 설정창 (LicenseDialog) Alt+G → settings_window_geometry 저장
 - 설정창 (CreditsDialog) Alt+G → settings_window_geometry 저장
 - 등록창 Alt+G → registration_window_geometry 저장
- korean_filename_normalizer
 - 메인창 Alt+G → window_geometry_override 저장
 - 설정창 Alt+G → settings_window_geometry 저장
 - 등록창 Alt+G → registration_window_geometry 저장
- attribute_reset
 - 메인창 Alt+G → window_geometry_override 저장
 - 설정창 Alt+G → settings_window_geometry 저장
 - 등록창 Alt+G → registration_window_geometry 저장

11. 테스트 결과 기록 템플릿

테스트 일시: YYYY-MM-DD HH:MM
 테스트 환경: Windows 11, Python 3.14.x
 빌드 버전: v0.x.x.x

- [] 1. 기본 실행 테스트
- [] 2. 신규 사용자 등록 플로우
- [] 3. 기존 사용자 (두 번째 앱)
- [] 4.1 유료 앱 크레딧 차감
- [] 4.2 무료 앱 동작
- [] 4.3 무제한 사용자 동작
- [] 5. 설정 파일 경로 검증

- [] 6. 오프라인 동작
- [] 7. 앱별 핵심 기능
- [] 8. Geometry 저장 기능 (Alt+G)

비고:

로드맵

Phase 1: 배포 준비 (완료) ☒

- ☒ bom_exporter 패키징 완료
- ☒ dwg_batch_print 패키징 완료
- ☒ dwg_classifier 통합 spec + NSIS + portable 완성
- ☒ conversion_verifier 통합 spec + NSIS + portable 완성
- ☒ korean_filename_normalizer 통합 spec + NSIS + portable 완성
- ☒ attribute_reset 패키징 완료
- ☒ 6개 앱 개별 빌드 스크립트 생성 (각 앱 폴더)
- ☒ 전체 앱 일괄 빌드 스크립트 (build_all.ps1)
- ☒ 모든 앱 README 문서화 완료
- ☒ 성능 최적화 완료 (3초 이내 로딩 목표 달성)
- ☒ 로딩 시간 테스트 스크립트 (test_loading_time.ps1)
- ☒ Git 저장소 초기화

Phase 2: 데모 준비 (완료) ☒

- ☒ 6개 앱 표준화 완료
- ☒ 빌드 실행 및 테스트 (build_all.ps1)
- ☒ NSIS 인스톨러 생성 자동화
- ☒ 포터블 버전 자동 패키징
- ☒ 성능 테스트 완료 (전체 PASS)

Phase 3: 배포 테스트 ☐

- ☐ 배포 형상 테스트 시나리오 실행
- ☐ 신규 사용자 등록 플로우 검증
- ☐ 크레딧 시스템 검증 (무료/유료/무제한)
- ☐ Google Sheets 동기화 검증
- ☐ 최종 배포

앱 아키텍처 및 구조적 일관성

앱 초기화 흐름 (Initialization Flow)

모든 6개 앱은 동일한 초기화 패턴을 따릅니다:

```

flowchart TD
    A[main 진입점] --> B[_acquire_single_instance  
뮤텍스 획득]
    B --> |실패| C[이미 실행 중 메시지  
종료]
    B --> |성공| D[_set_cross_app_running  
wf_rpa_config.json 기록]
    D --> E[Tk root 생성]
    E --> F[GUI 클래스 생성  
__init__]
    F --> G[_load_settings_fast  
설정 로드]
    G --> H[_setup_ui_basic  
UI 기본 구성]
    H --> I[_start_background_init  
백그라운드 초기화 시작]
    I --> J[_load_policies_async  
정책 비동기 로드]
    J --> K[_init_credit_manager_async  
크레딧 매니저 초기화]
    K --> L[mainloop 시작]
    L --> M{창 닫기?}
    M --> |Yes| N[_on_closing  
종료 핸들러]
    M --> |No| O[_sync_credit_async  
크레딧 동기화]
    O --> P[_clear_cross_app_running  
실행 상태 해제]
    P --> Q[종료]

```

GUI 라이프사이클 (GUI Lifecycle)

```

stateDiagram-v2
    [*] --> Initializing: main() 호출

    Initializing --> Loading: __init__ 시작
    Loading --> Ready: UI 완전 로드

    Ready --> AdminMode: 관리자 비밀번호 입력
    AdminMode --> Ready: 관리자 모드 종료

    Ready --> Working: 작업 시작 버튼
    Working --> Ready: 작업 완료/취소

    Ready --> Settings: 설정 열기 (Alt+G)
    Settings --> Ready: 설정 닫기

    Ready --> Closing: 창 닫기
    Closing --> [*]: 프로세스 종료

    note right of AdminMode

```



```

- 로그 패널 표시
- HW 정보 로깅
- 디버그 기능 활성화
end note

note right of Working
- 크레딧 차감
- 진행률 표시
- 취소 가능
end note

```

모듈 구조 (Module Structure)

```

graph TB
    subgraph "앱 레이어 (App Layer)"
        A1[ui_main.py  
GUI + 라이프사이클]
        A2[automation.py  
핵심 작업 로직]
        A3[app_setting_data.py  
설정 관리]
    end

    subgraph "공통 레이어 (Common Layer)"
        C1[wf_credit_manager.py  
크레딧 관리]
        C2[wf_register.py  
사용자 등록]
        C3[wf_log.py  
통합 로깅]
        C4[wf_goolesheets_manager.py  
Sheets 연동]
        C5[wf_settings_common.py  
설정 유틸리티]
        C6[wf_hwinfo.py  
HW 정보 수집]
    end

    subgraph "데이터 레이어 (Data Layer)"
        D1[wf_rpa_config.json  
전역 설정]
        D2[policy.json  
크레딧 정책]
        D3[settings.json  
앱 설정]
        D4[credit_history.json  
사용 이력]
    end

    A1 --> C1
    A1 --> C2

```

A1 --> C3
A1 --> A3
A2 --> C3
A3 --> C5

C1 --> C4
C1 --> D2
C1 --> D4
C2 --> C4
C2 --> C6
C2 --> D1
A3 --> D3

앱별 기능 일관성 매트릭스 (Feature Consistency Matrix)

기능	BE	DP	DC	CV	KFN	AR	상태
초기화							
main() 진입점	☑	☑	☑	☑	☑	☑	통일
단일 인스턴스 락 (Mutex)	☑	☑	☑	☑	☑	☑	통일
Cross-app 실행 체크	☑	☑	☑	☑	☑	☑	통일
실행 모드 감지	☑	☑	☑	☑	☑	☑	통일
GUI							
Tk 기반 UI	☑	☑	☑	☑	☑	☑	통일
Geometry 저장 (Alt+G)	☑	☑	☑	☑	☑	☑	통일
캡처 단축키 (Alt+C)	☑	☑	☑	☑	☑	☑	통일
창 최소 크기 제한	☑	☑	☑	☑	☑	☑	통일
Geometry 저장 (창별)							
메인창 geometry 저장	☑	☑	☑	☑	☑	☑	통일
설정창 geometry 저장	☑	☑	☑	☑	☑	☑	통일
등록창 geometry 저장	☑	☑	☑	☑	☑	☑	통일
크레딧 시스템							
Lazy 스레드 초기화	☑	☑	☑	☑	☑	☑	통일
종료 시 동기화	☑	☑	☑	☑	☑	☑	통일
크레딧 상태 표시	☑	☑	☑	☑	☑	☑	통일
관리자 모드							
비밀번호 인증	☑	☑	☑	☑	☑	☑	통일

기능	BE	DP	DC	CV	KFN	AR	상태
로그 패널	☑	☑	☑	☑	☑	☑	통일
HW 정보 로깅	☑	☑	☑	☑	☑	☑	통일
dev 모드 비밀번호 우회	☑	☑	☑	☑	☑	☑	통일
프로파일링							
Startup 프로파일링	☑	☑	☑	☑	☑	○	통일
버퍼 방식 로깅	☑	☑	☑	☑	☑	○	통일
설정 시스템							
app_setting_data.py	☑	☑	☑	☑	☑	☑	통일
설정 창 (Toplevel)	☑	☑	☑	☑	☑	☑	통일

범례:

- ☑ 구현됨 / ✕ 미구현 / ○ 해당없음 (경량 앱)

실행 모드 감지 패턴 (Run Mode Detection)

모든 앱에서 통일된 `_detect_run_mode()` 함수 사용:

```
def _detect_run_mode():
    """
    실행 모드 감지 (환경변수 + sys.argv 기반 통일 방식)
    - 1순위: WF_RPA_MODE 환경변수 (demo 모드 명시적 지정용)
    - 2순위: .py 파일 직접 실행 → dev
    - 3순위: 기본값 release (exe 실행)
    """
    env_mode = (os.environ.get("WF_RPA_MODE") or "").strip().lower()
    if env_mode in ("dev", "demo", "release"):
        return env_mode
    if sys.argv[0].endswith(".py"):
        return "dev"
    return "release"
```

모드별 동작 차이 (Mode-specific Behavior)

```
flowchart LR
    subgraph dev ["DEV 모드"]
        D1[설정 경로: 10.common/config/]
        D2[관리자 비밀번호 우회]
        D3[콘솔 로그 출력]
        D4[Startup 프로파일링 활성화]
    end
```

```

subgraph demo ["DEMO 모드"]
  M1[설정 경로: 10.common/config/]
  M2[관리자 비밀번호 필요]
  M3[데모용 크레딧 제한]
  M4[프로파일링 비활성]
end

subgraph release ["RELEASE 모드"]
  R1[설정 경로: ~/.wf_rpa/]
  R2[관리자 비밀번호 필요]
  R3[정상 크레딧 동작]
  R4[프로파일링 비활성]
end

```

설정 파일 경로 해석 (Settings Path Resolution)

```

flowchart TD
  A[_detect_run_mode] --> B{모드?}

  B --> |dev/demo| C[10.common/config/]
  B --> |release| D[~/.wf_rpa/]

  C --> E[wf_rpa_config.json]
  전역 설정]
  C --> F[{app}/policy.json]
  크레딧 정책]
  C --> G[{app}/settings.json]
  앱 설정]

  D --> H[wf_rpa_config.json]
  전역 설정]
  D --> I[{app}/policy.json]
  크레딧 정책]
  D --> J[{app}/settings.json]
  앱 설정]

  subgraph fallback ["Fallback (release)"]
    K[_internal/.wf_rpa/ 번들 복사]
  end

  D --> |파일 없음| K
  K --> H
  K --> I
  K --> J

```

일관성 점수 (Consistency Score)

현재 6개 앱의 구조적 일관성 점수: **100%**

모든 일관성 항목 달성:

- ☒ 실행 모드 감지 패턴 (6/6)
- ☒ 단일 인스턴스 락 (6/6)
- ☒ 관리자 모드 구현 (6/6)
- ☒ 설정 관리 시스템 (6/6)
- ☒ 크레딧 시스템 통합 (6/6) - 모두 Lazy 스레드 방식
- ☒ Startup 프로파일링 (5/5, AR 제외-경량앱)
- ☒ 키보드 단축키 Alt+G/Alt+C (6/6)

🔑 구성 표준화 및 정리 로그

변경 요약

- 모든 앱이 동일한 구성 표준을 사용합니다: 전역 `~/.wf_rpa/wf_rpa_config.json` + 앱별 `~/.wf_rpa/{app}/{settings.json, credit_policy.json, credit_history.json}`.
- 개발 환경에서는 동일 구조가 각 앱 폴더의 `config/` 하위에 위치합니다.
- Google Sheets 서비스 계정 자격증명은 다음 위치에서 탐색됩니다:
 - 개발: `{app_folder}/config/.silver-argon-*.json`
 - 배포: `C:\\Users\\{username}\\\\.wf_rpa\\.silver-argon-*.json`
- 구 레거시 정책 파일 `wf_app_policies.json`은 더 이상 사용하지 않으며 `credit_policy.json`으로 대체되었습니다.
- `10.rpa/config` 폴더는 제거되었습니다. 각 앱 폴더의 `config/` 폴더를 사용합니다.

실제 정리 작업

- 다음 경로의 레거시 `wf_app_policies.json`을 제거했습니다:
 - `10.rpa/30.apps/bom2excel/config/wf_app_policies.json` (제거 완료)
 - `10.rpa/50.data/conversion_verifier/config/wf_app_policies.json` (제거 완료)
 - `10.rpa/50.data/dwg_classifier/config/wf_app_policies.json` (제거 완료)
- `10.rpa/config` 폴더 제거 완료 (2025-11-29)
- `10.rpa/release` 폴더 제거 완료 (2025-11-29)

앱별 메모

- **Korean Filename Normalizer (kfn)**
 - 기본 경로: 사용자 Downloads (Known Folder API)
 - "새폴더로 저장" 옵션은 설정으로 이동, 메인에는 "현재 폴더 스캔" 버튼과 자동 스캔 동작.
 - 스캔 대상 계산은 자소 분리 파일만 집계, 결과 팝업에서 [번호 | 자소분리파일 | 복원파일] 테이블 제공.
 - 무료 앱 (체험판 크레딧 -1)
- **Conversion Verifier (cv)**
 - `app_setting_data.py`를 도입해 표준 로더로 전환, 구형 배치/숨김 파일 배포 로직 제거.
 - 홈 루트(`~/.wf_rpa`)의 레거시 숨김 파일이 감지되면 표준 경로로 1회 마이그레이션합니다.
 - 무료 앱 (trial_credits: -1)
- **DWG Classifier (dc)**

- 개발 환경에서 서비스 계정 자격증명을 `50.data/dwg_classifier/config/`에 배치하여 Sheets 경고를 제거했습니다.

- **BOM Exporter (be)**

- 가장 무거운 앱 (SolidWorks 자동화)
- 크레딧: 100 크레딧/파일

- **Attribute Reset (ar)**

- DWG 파일 속성 일괄 초기화
- 크레딧: 200 크레딧/작업

빠른 점검 체크리스트

1. 각 앱에서 설정 저장/재시작 시 `settings.json`이 표준 경로에 반영되는지 확인
2. 크레딧 차감/동기화 동작 시 `credit_history.json` 업데이트 확인
3. 정책 변경 시 `credit_policy.json` 반영 및 앱 재시작 후 적용 확인
4. Google Sheets 연동 경고가 출력되지 않는지 확인 (개발: per-app, 배포: 전역 경로)

문서 체계

변경 이력 (CHANGELOGs)

프로젝트는 책임 범위별로 분리된 CHANGELOG를 유지합니다:

- **CHANGELOG.md** - 인프라/빌드 시스템 변경사항
 - PyInstaller 설정, 빌드 스크립트, spec 파일 구조
 - 디렉토리 구조 변경, 번들링 정책
- **10.common/CHANGELOG.md** - 공통 모듈 변경사항
 - `wf_credit_manager`, `wf_googlesheets_manager` 등 공통 라이브러리
 - 정책 시스템, 크레딧 로깅, 환경 감지 로직
- **앱별 CHANGELOGs** - 앱 기능 변경사항
 - [30.apps/bom2excel/CHANGELOG.md](#)
 - [50.data/dwg_classifier/CHANGELOG.md](#)
 - [50.data/conversion_verifier/CHANGELOG.md](#)
 - [50.data/korean_filename_normalizer/CHANGELOG.md](#)

개발 문서 (docs/)

- **docs/TROUBLESHOOTING.md** - 빌드 및 실행 오류 해결
- **docs/DEPLOYMENT.md** - 배포 가이드
- **docs/CODING_STANDARDS.md** - 코딩 표준/네이밍 규칙
- **docs/UI_STRUCTURE.md** - 앱별 UI 구조 요약
- **docs/STATIC_ANALYSIS.md** - 정적 분석/품질 도구 계획
- **docs/TEST_PLANS.md** - 테스트 플랜 통합 문서

히스토리 문서 (docs/history/)

과거 개발 로그 및 완료된 이슈 보고서는 [docs/history/](#) 폴더에 아카이브되어 있습니다:

- [2025-10-18_dev_log.md](#), [2025-11-15_dev_log.md](#) 등 - 일자별 개발 일지
 - [console_flash_fix.md](#), [credit_logging.md](#) 등 - 완료된 이슈 보고서
-

✉ 연락처

- **Email:** insung.lee@worksfree.co.kr
 - **Organization:** WorksFree Co., Ltd.
 - **Website:** <https://worksfree.co.kr>
-

참고 문서

- [10.common/ReadMe.md](#): 공통 모듈 상세 설명
- 각 앱별 README.md: 앱별 사용 가이드 및 아키텍처 설명
 - [bom2excel/README.md](#)
 - [dwg_classifier/README.md](#)
 - [korean_filename_normalizer/README.md](#)
 - [conversion_verifier/README.md](#)